

The extended raising ladder $b^\dagger S_{\mu-(k+1)/2} = \beta^{1/2} S_{\mu-k/2}$ and the extended number operator

Claude, with Daniel Murfet

2026-09-06

Abstract

The payoff of the smoothness foundation: the *raising* half of `lem:extended_ladder`. Where the lowering relation was definitional (unit 19), raising carries the analytic content — $b^\dagger$ climbs the ladder *against* the direction of the recursion, so it needs the Weyl commutator $[b, b^\dagger] = 1$ and hence C^∞ regularity of the extension. We prove $b^\dagger S_{\mu-(k+1)/2} = \beta^{1/2} S_{\mu-k/2}$ by induction on k , and combine it with the lowering relation to obtain the extended number operator $b^\dagger b S_{\mu-k/2} = (2(\mu - k/2) - 1) S_{\mu-k/2}$. This completes the ladder/log structure of grammar §4. Zero sorry/axiom.

1 Setting

Recall $\mathbf{Sneg} \beta \mu k = S_{\mu-k/2}$, defined by $S_\lambda = \frac{\sqrt{\beta}}{2\lambda} b S_{\lambda+1/2}$. The lowering relation $b S_{\mu-k/2} = (2(\mu - k/2) - 1) \beta^{-1/2} S_{\mu-(k+1)/2}$ is that definition rearranged. The raising relation is its inverse in the ladder; to prove it we exploit $b^\dagger b = b b^\dagger - 1$ (the commutator) and reduce $b^\dagger$ acting on a lowered function to $b^\dagger$ acting one rung up, where the inductive hypothesis (or, at the base, the integral-regime action $b^\dagger S_\mu = \beta^{1/2} S_{\mu+1/2}$) applies.

2 The things formalised

```
theorem raiseOp_Sneg (  : ) (h : 0 < ) (h : 0 < )
  (hpole : j : , 1 j → - (j : ) / 2 0) :
  k : , raiseOp (fun a => Sneg (k + 1) a)
    = fun a => ^ ((1:)/2) * Sneg k a

theorem number_operator_Sneg (  : ) (h : 0 < ) (h : 0 < )
  (hpole : j : , 1 j → - (j : ) / 2 0) (k : ) :
  raiseOp (lowerOp (fun a => Sneg k a))
    = fun a => (2 * ( - (↑k)/2) - 1) * Sneg k a
```

The non-pole hypothesis $\mu - j/2 \neq 0$ for all $j \geq 1$ says μ is not a positive half-integer — exactly the condition that keeps every prefactor $\frac{\sqrt{\beta}}{2(\mu-j/2)}$ finite along the descent.

3 Proof strategy

Induction on k for the raising relation. Both base and step share one shape: write $S_{\text{next}} = c \cdot b S_{\text{here}}$ (the `Sneg` recursion), pull the scalar out (`raiseOp_const_mul`), apply $b^\dagger b = b b^\dagger - 1$ (`raiseOp_lowerOp`), then substitute $b^\dagger S_{\text{here}}$ and lower once more:

- **Base** ($k = 0$): $b^\dagger S_\mu = \beta^{1/2} S_{\mu+1/2}$ (`raiseOp_fluctuation`) and $b S_{\mu+1/2} = 2\mu \beta^{-1/2} S_\mu$ (`lowerOp_fluctuation`), giving $b^\dagger b S_\mu = (2\mu - 1) S_\mu$; the prefactor $c_1 = \beta^{1/2}/(2\mu - 1)$ cancels it.
- **Step**: the inductive hypothesis gives $b^\dagger S_{\mu-k/2} = \beta^{1/2} S_{\mu-(k-1)/2}$ in disguise; `Sneg_lowering` lowers, and the numeric collapse $2(\mu - k/2) - 2 = 2(\mu - (k + 2)/2)$ matches the prefactor denominator.

In each case the $\beta^{\pm 1/2}$ factors meet as $\beta^{1/2}\beta^{-1/2} = 1$ and a `div_self` at the non-pole finishes. The number operator is then immediate: lower (`Sneg_lowering`), raise (`raiseOp_Sneg`), and the $\beta^{\pm 1/2}$ cancel, leaving the eigenvalue $2(\mu - k/2) - 1$.

4 Roadblocks and resolutions

- The whole computation is a `calc` over pointwise scalar identities, but the commutator `ladder_commutator` and the linearity helpers act on *functions* through `deriv`. The clean interface: state $b^\dagger(c \cdot g) = c b^\dagger g$ and $b(c \cdot g) = c b g$ as pointwise lemmas with a `DifferentiableAt` side-condition (discharged from `contDiff_Sneg`), and rearrange the commutator to $b^\dagger(bf) = b(b^\dagger f) - f$.
- Feeding a function equality (the inductive hypothesis, or `raiseOp_fluctuation/Sneg_lowering` lifted by `funext`) into a `deriv`-bearing operator requires it as a genuine function identity, then `rw`. Keeping everything in the $\lambda a. \text{Sneg } \beta \ \mu \ _ \ a$ shape lets `calc`'s `defeq` boundaries absorb the eta/unfolding steps of the `Sneg` recursion.
- The final scalar collapses do not close by `ring` alone — `ring` cannot see $\beta^{1/2}\beta^{-1/2} = 1$ (`rpow` addition). Group the powers by a `show ...by ring` rewrite, apply the $\hbar\beta\beta$ fact, then discharge the remaining $\frac{c}{d} \cdot d = c$ by `field_simp` with the denominator non-vanishing (supplied in the normalized $2\mu - (k+2) \neq 0$ form).
- Style-linter notes carried forward: a goal-changing `show` is rejected — use `change`; module-docstring lines still count toward the 100-column limit.

5 What was Mathlib, what was new

Mathlib gave only `deriv_const_mul`, `Real.rpow_add`, `field_simp`, `linear_combination`. Everything else is the grammar-§4 tower built this arc: the ladder operators and commutator (unit 16), the extension (unit 19), the smoothness foundation (previous unit), and here the raising induction plus its number-operator corollary.

6 Lessons

An operator recursion gives one ladder direction free and makes the opposite direction the theorem: raising is $b^\dagger b = b b^\dagger - 1$ applied once, with the inductive hypothesis supplying the one-rung-up value. Encode the operators' scalar-linearity as pointwise lemmas with `DifferentiableAt` obligations sourced from the smoothness unit, and let `calc` `defeq` soak up the recursion's unfolding.

7 Outcome

The entire ladder/log structure of grammar §4 — derivative ladder, recurrence, ODE, boundary, oscillator algebra $[b, b^\dagger] = 1$, log insertions and their ODE/generating function, the parabolic-cylinder closed form, Weber/Hermite/QHO, the regular case, the extension below zero, and now the extended raising ladder and number operator — is formalised, zero **sorry**/axiom.